

Scaling Trajectories for Complex Tasks through Recursive Self-Rewrite

Zongxia Li^{†1,2}, Yucheng Shi^{†1}, Zhongzhi Li^{†1,3}, Junyao Yang^{1,4}, Ruhan Wang^{1,5}, Chongsong Huang⁶, Fuxiao Liu², Haitao Mi¹, Jordan Boyd-Graber⁷, LeoweiLiang¹

¹Tencent HY LLM Frontier ²University of Maryland, College Park ³University of Georgia ⁴National University of Singapore ⁵Indiana University

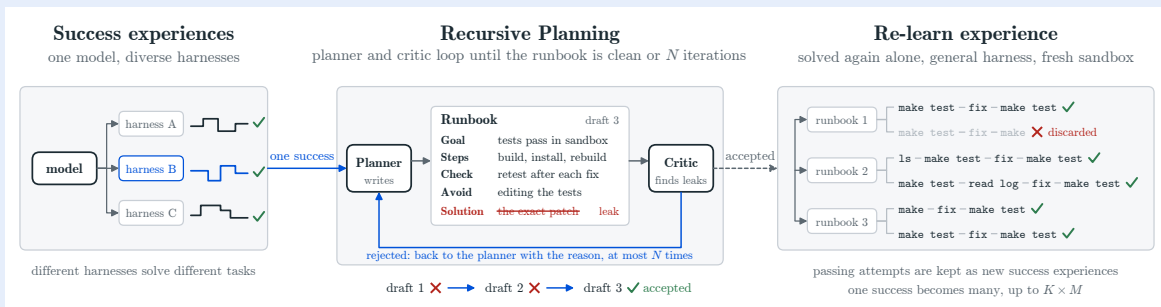
⁶Washington University in St. Louis ⁷Nanyang Technological University

[†] Core contribution.

zli12321@terpmail.umd.edu, zhongzhili@global.tencent.com

Successful trajectories on difficult tasks are valuable supervision for model improvement. Different harnesses enable the same model to solve these tasks in different ways. Their successful trajectories provide useful experience for self-improvement, but they also contain controller interventions and workflow conventions that may be unavailable under a general harness. We propose RECURSIVE SELF-REWRITE (RSR), a framework that converts these experiences into reusable model capabilities through recursive trajectory self-rewrite. We use a single base model, Qwen-3.8-27B to discover successful solutions under diverse harnesses and rewrite them for learning under a general harness. RSR contains a **planner** that extracts useful procedures into runbooks, a **critic** that screens for verifier and solution leakage, and rejects candidates and recursively regenerate using critique feedback, and an **executor** that follows the qualified runbooks to solve each task in a fresh sandbox under a general harness. We show that using three harnesses expands the range of task domains that Qwen-3.8-27B can solve and yields more valuable successful trajectories; with RSR, we further reconstruct these into a larger set of high-quality trajectories. We collect experience from approximately 3K self-curated terminal tasks. Across 3K tasks, the union of the three harnesses solves 759 tasks, which is 34.3% more than the strongest individual harness in the recorded pool. We further use RSR to rewrite successful source trajectories, expanding the training set from 2,001 to 11,094 high-quality trajectories for finetuning Qwen-3.8-27B. Training it on these rewritten trajectories outperforms both the base model and direct trajectory SFT: pass@3 increases from 57.0% to 74.2% on Terminal-Bench 2, from 1.5% to 9.1% on Terminal-Bench 4, from 39.0% to 63.0% on our self-curated Terminal-Bench Hard, and from 3.0% to 6.0% on our Software Terminal-Bench. On Long-Horizon Terminal-Bench, process reward rises from 0.21 to 0.29.

 <https://huggingface.co/IntelligenceLab/RSR-27B>



Recursive planning turns successful experience into new practice. Diverse harnesses discover source successes. Planner-critic feedback refines runbooks within a retry budget; each screened runbook guides fresh executions under a general harness. Rewritten trajectories are verified and filtered before training. Commands and draft numbers are schematic examples.

Date: October 5, 2026

1 Introduction

Human progress advances by turning experience into knowledge that others can reuse and by discovering new solutions. Researchers approach problems with different tools, methods, and prior knowledge, producing successes, failures, and practical lessons. These lessons are organized into papers and textbooks so that later generations can learn from them without repeating the entire discovery process. We argue that AI agents can undergo a similar process through self-improvement, turning successful experience into reusable capability.

A model’s capabilities depend not only on the model itself, but also on the harness through which it acts. A harness determines how the model receives observations, invokes tools, tracks progress, verifies intermediate results, recovers from failure, and decides when to stop [36]. A growing body of work shows that improved harnesses can substantially increase terminal-task performance without updating model parameters [8, 9, 14]. In some settings, stronger harnesses even enable weaker models to match or surpass stronger models, as demonstrated on terminal benchmarks and game-playing tasks [16, 26]. Different harnesses encourage different behaviors and are often effective for different task domains [4, 32, 36]. For example, coding-oriented harnesses such as Codex [22] and Claude Code [1] are designed for coding tasks, goal-driven loops promote long-horizon exploration through repeated continuation [2], and StateM structures execution around explicit states and validated transitions [26]. As a result, different harnesses can solve complementary subsets of tasks, and access to multiple harnesses expands the set of tasks a model can solve (Section 3.3). When a model solves additional tasks under different harnesses, the resulting successful trajectories reveal behaviors and strategies that would not emerge under a single general-purpose harness, which these trajectories provide supervision for improving the model itself.

However, naively training on trajectories collected from different harnesses can introduce substantial distribution mismatch. Each trajectory reflects the joint behavior of the model and its source harness, including controller interventions, task-specific prompts, workflow logic, and stopping rules. Simply pooling these trajectories can mix harness-specific conventions rather than isolate the underlying problem-solving behaviors that made the solutions successful [30]. As a result, a model trained on such data may rely on external control patterns that will be unavailable under a general harness at inference time. Effective self-improvement requires preserving useful planning, verification, recovery, and continuation behaviors while rewriting them into trajectories that are compatible with the general harness the model will ultimately use.

We propose RECURSIVE SELF-REWRITE, a framework for self-improvement through multi-harness trajectory rewriting into a general harness, as shown in the opening illustration. RECURSIVE SELF-REWRITE first uses distinct harnesses to collect more diverse successful solutions across complementary parts of the task distribution. It then rewrites each solution using the same base model in three roles: a **planner** reconstructs the solution as a runbook that captures the key steps, checks, and recovery strategies; a **critic** removes verifier leakage and solution leakage, and an **executor** then follows the accepted runbook to solve the task again in a fresh sandbox under the general Terminus 2 harness [5]. We generate multiple runbooks and executions for each source solution and retain only passing trajectories through rejection sampling. The entire pipeline uses a single base model, Qwen-3.8-27B [27] end-to-end.

We use an internal RST pipeline [11] to automatically curate $\sim 3K$ difficult terminal tasks, which spans 50+ domains including e.g. Science, Software, Hardware, Data. We use Terminus 2, StateM [26], and Recursive Self-Reflect Terminus (RSRT) as discovery harnesses to collect approximately 2K successful trajectories covering 759 unique tasks. These harnesses solve complementary subsets of tasks and broadens the range of successful experiences available for solving tasks. We then apply RECURSIVE SELF-REWRITE with multiple executions per passing trajectory and generate approximately $\sim 10K$ verified trajectories for supervised finetuning. Our results show that training on the rewritten trajectories improves Qwen-3.8-27B over Direct SFT by 20.8, 4.1, 4.6, 7.0, and 3.0 percentage points in pass@3 on Terminal-Bench 2 (TB2), Terminal-Bench 3 (TB3), Terminal-Bench 4 (TB4), Terminal-Bench Hard (TBH), and our Software Terminal Bench 100 (SWR 100), respectively. Mean per-run pass rates also improve on all five benchmarks, while process reward on Long-Horizon Terminal Bench (LHTB) increases from 0.25 to 0.29. RECURSIVE SELF-REWRITE show that different harnesses are not only useful as inference-time controllers, but also as tools for discovering experiences through which a model

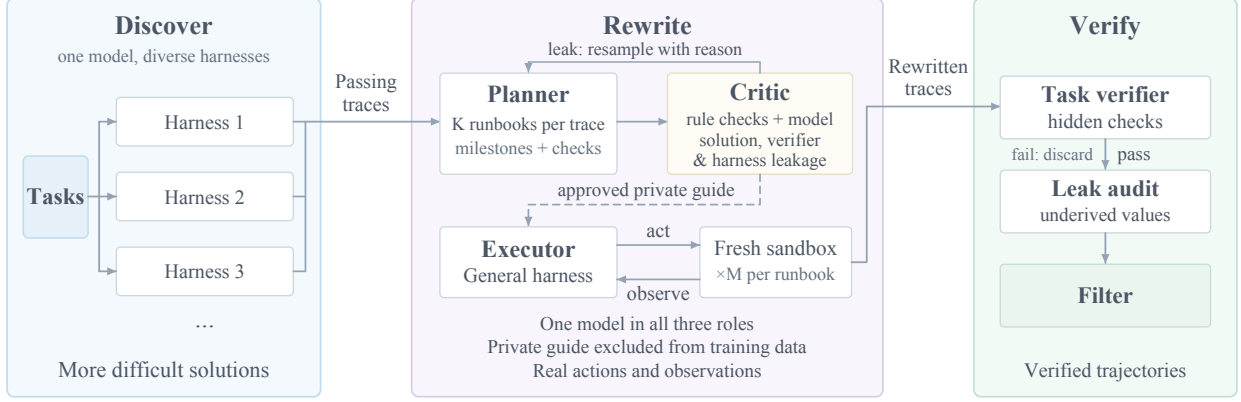


Figure 1 Turning multi-harness successes into learning experiences with Recursive Self-Rewrite. Different discovery harnesses enable the same model to solve more difficult tasks. RECURSIVE SELF-REWRITE rewrites these harness-assisted solutions into demonstrations under a general harness, allowing the model to learn from successful experiences that a single harness alone would not produce.

can improve its own general-purpose policy.

2 Recursive Self-Rewrite: Self-Improvement Through Trajectory Rewriting

RECURSIVE SELF-REWRITE improves a model under a general-purpose harness by learning from successful trajectories discovered under diverse specialized harnesses. The key idea is that different harnesses help the same base model solve different difficult tasks, and those successful solutions can be rewritten into demonstrations that are compatible with a single general harness. We use the same base model throughout the process in three stages. First, we run the base model under multiple discovery harnesses and collect successful trajectories. Second, we rewrite each successful trajectory into new demonstrations under the general harness using three model roles: a **planner**, a **critic**, and an **executor**. Then we keep only verified rewritten trajectories as the training sources for model self-improvement. Figure 1 illustrates the overall process.

2.1 Multi-Harness Discovery

We use multiple discovery harnesses that differ in how they provide control and support during execution, such as progress tracking, continuation, validation, state management, or recovery from failure. Because different harnesses can be effective on different types of tasks, they expand the range of successful solutions where the model itself with just one harness can solve.

2.2 Trajectory Rewriting for Experience Learning

Successful trajectories collected under different harnesses record how the model solves tasks with different workflow logic. Our goal is to turn these successful solutions into learning experiences that the model can practice and learn from under a general harness. Intuitively, discovery helps the model solve a task once, while rewriting helps the model solve it again in a cleaner and more reusable way. Trajectory rewriting involves three roles: a **planner** reconstructs the successful procedure as a runbook, a **critic** removes leakage and harness-specific information, and an **executor** follows the guidance to solve the task again under the general harness. The resulting rewritten trajectories provide cleaner demonstrations through which the model can relearn the solution and improve its own policy.

Planner: Reconstructing Successful Solutions into Plans. The planner reconstructs a *runbook* for each successful source trajectory: a structured description of how the task was solved. Before planning, we compact the source trajectory by keeping the task instruction, the model’s actions, and the environment’s observations, while removing harness-specific control messages. The runbook summarizes the required end state, key milestones, useful checks, recovery strategies, and common pitfalls. To reduce direct answer transfer, runbooks may describe the task, relevant interfaces, and validation procedures, but they should not directly provide the finished deliverable. We sample multiple runbook candidates for each source trajectory.

Critic: Filtering Leakage. We use the model itself as a critic to filter candidate runbooks before they are used for execution. We first apply deterministic checks, such as schema validation, removal of known artifacts, and rejection of unsupported tool references. We then use a model-based critic that sees only the public task instruction and the candidate runbook. Its role is to determine whether the runbook provides useful procedure or leaks information that the executor should not receive. Only runbooks that pass this screening stage are kept for rewriting.¹

Executor: Relearning Successful Behaviors Under the General Harness. For each runbook, the executor re-solves the task in a fresh sandbox under the general harness. The runbook is provided as private guidance during generation, but it is never written into the public trajectory. The executor must therefore produce a new trajectory based on the current environment rather than replaying the source trajectory.

Trajectory Filtering. We use the model itself to flag values that appear in a trajectory but cannot be derived from the task or the environment, a sign of hidden answer transfer, and discard the demonstrations that contain them. For finetuning, we keep only the public interaction history (the task instruction, environment observations, and the model’s responses) and remove the runbook and critic conversation. At inference time, the finetuned model runs under the general-purpose harness alone, without the source harnesses or private runbooks, so any planning, checking, recovery, or continuation must come from the model itself. RECURSIVE SELF-REWRITE thus turns diverse harness-assisted successes into reusable experience for a single general-purpose harness.

3 Experiments

We organize the experiments around the stages of self-improvement: collecting experience through diverse harnesses, reconstructing and auditing that experience, and training and evaluating the resulting model.

3.1 Terminal Tasks

We construct our training task pool from two sources. **SWR** is our self-constructed collection of approximately 2.5K terminal tasks spanning across 50 domains. These domains include software usage, biology, chemistry, physics, hardware, operations, and security. This breadth provides diverse settings in which the model must select tools, reason about environment feedback, and carry out task-specific procedures. We additionally include 420 filtered and modified tasks from the **RST** [11] dataset with increased difficulty. These sources define the approximately 3K-task pool.

3.2 Discovery Harnesses

We use three harnesses with different approaches to terminal problem solving: Terminus 2 [5], StateM [26], and our modified Terminus 2–Recursive Self-Reflect Terminus (RSRT). The underlying model is held fixed across harnesses, so the diversity of collected experience comes from differences in external execution support (Figure 2).

¹This part can be done with Jev for faster inference [3]

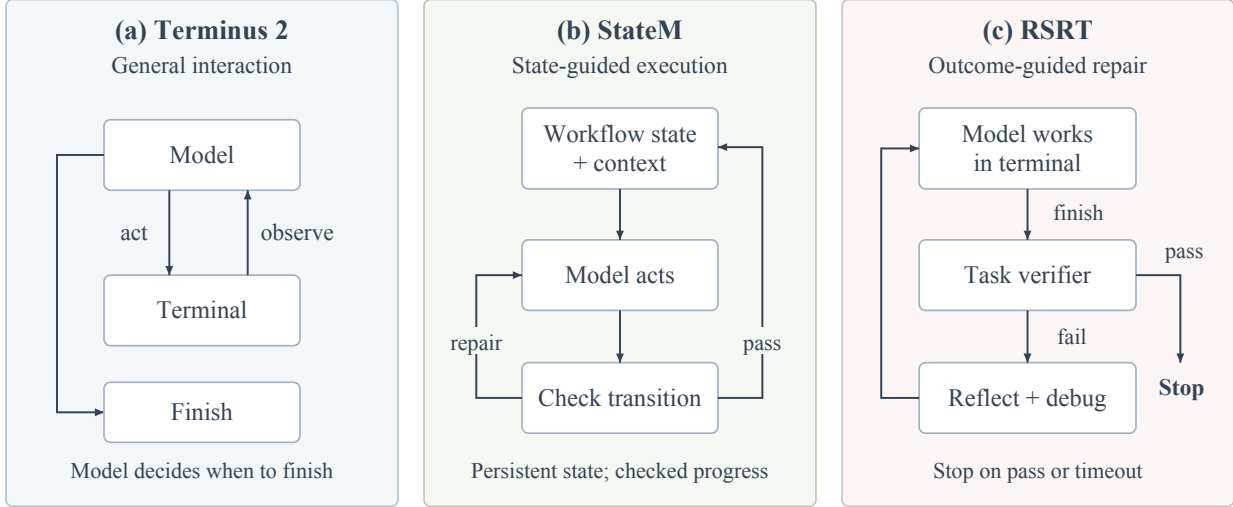


Figure 2 Three harnesses represent different logics of solving problems. Terminus 2 supports a general model–terminal interaction loop. StateM adds persistent workflow state and checked transitions. Recursive Self-Reflect Terminus uses verifier outcomes to trigger reflection and continued execution, stopping on success or timeout.

Terminus 2: general terminal interaction. Terminus 2 provides our general-purpose baseline harness [5]. The model issues terminal commands, observes their outputs, and decides how to proceed or when to finish. It provides a common interaction interface for collecting trajectories with relatively little specialized workflow control, and is also the target harness used for reconstructed executions. This is also the general harness we use for model inference.

StateM: explicit workflow control. StateM organizes execution around persistent workflow states, phase-local context, and checked transitions [26]. Recoverable runbooks and procedural practices help preserve progress and guide continuation after interruptions or failures. This structure supports problem solving in which the model must maintain execution state, satisfy intermediate conditions, and verify progress before moving to the next phase.

Recursive Self-Reflect Terminus (RSRT) Recursive Self-Reflect Terminus extends Terminus 2 to continue problem solving after unsuccessful completion attempts without revealing verifier internals or reference solutions, using only the final verification outcome as weak supervision. The discovery protocol allows a wall-clock budget of up to 180 minutes per task. When the model declares completion, the harness runs the task verifier on the current result. A passing result ends the rollout. If verification fails and time remains, the harness reports only the failure outcome and prompts the model to review its previous trajectory, identify possible mistakes, debug its work, and continue. This process repeats until the task passes or the time limit is reached. The design encourages reflection and recovery beyond the model’s initial stopping decision while keeping the verifier and the solution hidden. Within a discovery rollout, continuation preserves the conversation and sandbox state. Trajectory reconstruction instead starts a new execution in a fresh sandbox under the general harness.

3.3 Multi-Harness Discovery and Rollouts

We use **Qwen-3.8-27B** to generate rollouts using three harnesses **Terminus 2**, **Recursive Self-Reflect Terminus**, and **StateM**. Then analyze the rollout domain coverage rates and pass rates by different harnesses. We pool passing trajectories across the three harnesses as candidates for runbook reconstruction and fresh execution under the general harness. This collection stage provides experiences generated through different approaches

Harness	RST (420)		SWR (2,509)		Pooled (2,929)	
	Solved	Unique	Solved	Unique	Solved	Unique
Terminus 2	94 (22.4%)	8	471 (18.8%)	121	565 (19.3%)	129
RSRT	108 (25.7%)	14	404 (16.1%)	77	512 (17.5%)	91
StateM	95 (22.6%)	10	366 (14.6%)	58	461 (15.7%)	68
Terminus 2 $\cup$ RSRT	119 (28.3%)	34	572 (22.8%)	264	691 (23.6%)	298
Terminus 2 $\cup$ StateM	115 (27.4%)	21	553 (22.0%)	226	668 (22.8%)	247
RSRT $\cup$ StateM	121 (28.8%)	35	509 (20.3%)	159	630 (21.5%)	194
All three	129 (30.7%)	+21	630 (25.1%)	+159	759 (25.9%)	+194

Table 1 The union of three harnesses lead to the most unique tasks solved. *Unique* denotes tasks solved by only that harness. For the final row, “+” denotes the additional tasks solved relative to the strongest individual harness. A task is counted as solved when at least one recorded rollout passes. Rollout counts differ across harnesses in the full pool due to sandbox errors or task errors.

Harness	Rollouts	Passing	Success rate	Tasks solved
Terminus 2	5,405	766	14.2%	565
RSRT	3,777	599	15.9%	512
StateM	5,416	636	11.7%	461
Pooled	14,598	2,001	13.7%	759

Table 2 Discovery rollouts on the terminal task set. Passing counts successful trajectories. Tasks solved counts distinct tasks; the pooled row uses the union across harnesses. Success rate is per rollout.

to the same training problems.²

We generated a total of 14,598 rollouts from the dataset, including 2,001 successful trajectories covering 759 distinct tasks. We distinguish *task coverage*, the fraction of tasks solved by at least one recorded rollout, from *rollout success rate*, the fraction of individual attempts that pass. Multiple passing trajectories can correspond to the same task.

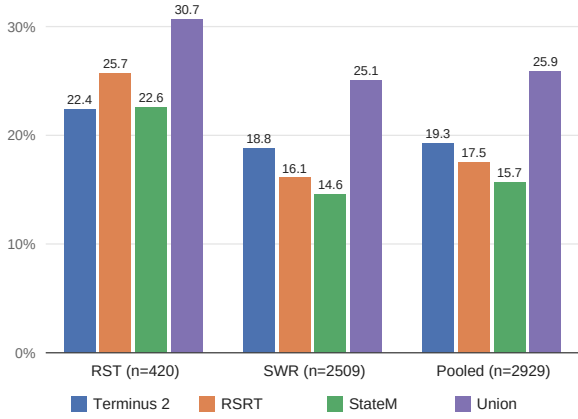
Using different harnesses expands the model’s successful solution coverage. Table 1 shows the tasks solved by each harness and by their unions. The three-harness union solves a total of 759 tasks (25.9%), adding 194 solved tasks with a relative increase of 34.3%. The union improves over the strongest individual harness by 19.4% on RST and 33.8% on SWR. These additional solved tasks broaden the experiences available and available successful trajectories for trajectory reconstruction.

Among the 759 tasks solved by at least one harness, 288 are solved exclusively by one harness: 129 only by Terminus 2, 91 only by Recursive Self-Reflect Terminus, and 68 only by StateM. Another 163 are solved by exactly two harnesses, while 308 are solved by all three. Every pairwise union exceeds the coverage of the strongest individual harness, and each third harness contributes further solved tasks (Figure 3).

Complementarity persists with equal rollout counts. The full pool contains 5,405 Terminus 2, 3,777 Recursive Self-Reflect Terminus, and 5,416 StateM rollouts, so its individual-harness ranking is affected by unequal sampling (Table 2). We therefore also examine the 1,247 tasks with equal rollout counts across harnesses: 420 RST tasks with one rollout per harness and 827 SWR tasks with two, totaling 2,074 rollouts per harness. On this subset, Terminus 2, Recursive Self-Reflect Terminus, and StateM solve 249 (20.0%), 285 (22.9%), and 227 (18.2%) tasks, respectively. Their union solves 352 (28.2%), adding 67 tasks over Recursive Self-Reflect Terminus, a relative increase of 23.5%. Each harness still contributes exclusive solves: 30, 56, and 21.

²Due to harness complexities of harness other than the general base harness, we have more sandbox errors rolling out from Recursive Self-Reflect Terminus and StateM, leading to fewer rollouts on Recursive Self-Reflect Terminus than other harnesses.

(a) Tasks solved: single harness vs. union



(b) Task overlap (pooled, exclusive regions)

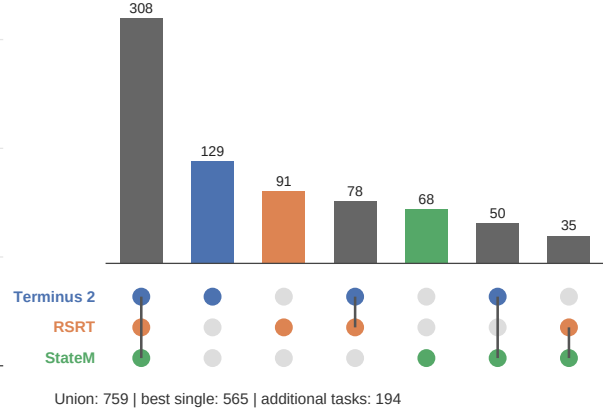


Figure 3 Complementary task coverage across harnesses. (a) Task coverage of each harness and their union on our terminal tasks, and the pooled task set. (b) Exclusive and overlapping solve regions on the pooled set. Colored dots indicate the harnesses included in each region. Of the 2,929 tasks, 2,170 are not solved by any harness. Coverage uses all recorded rollouts; per-harness rollout counts differ.

Different harnesses complement different domain problems. We analyze the domain and task-type coverage from the three harnesses. The dataset contains 87 fine-grained labels: 35 RST task categories and 52 SWR tool families. These labels differ from the broad scientific and engineering domains. Under this taxonomy, Terminus 2, RSRT, and StateM solve at least one task in 49, 46, and 42 domains, respectively, whereas their union reaches 51. Across 88 task types, they cover 56, 52, and 49 types individually, while their union covers 60 domains.

Figure 4 shows that no individual harness has the highest observed coverage in every domain. RSRT leads on RST scripting and automation (35.7%) and SWR media and signal processing (13.1%). StateM leads on RST software development and debugging (26.2%), while Terminus 2 leads on RST version control and several SWR scientific domains. Combining harnesses raises SWR media and signal coverage to 22.3% and earth and geospatial coverage to 19.8%, compared with best individual rates of 13.1% and 12.2%.

Successful discovery trajectories differ in execution style. The 2,001 passing trajectories also exhibit different execution styles (Table 3). Terminus 2 has a median of 17 turns and assigns 55.5% of commands to exploration. Recursive Self-Reflect Terminus has a median of 18 turns and averages 2.20 completion claims per trajectory; 4.5% of its passing trajectories contain a rejected completion before eventual success. StateM has a median of 24 turns and devotes 29.6% of commands to state-management operations. In the fixed sample of 200 failed trajectories per harness (100 from each data source), Recursive Self-Reflect Terminus averages 67.1 turns, compared with 22.8 for Terminus 2 and 23.4 for StateM. Thus, extended execution can recover some tasks while also spending substantial effort on unsuccessful attempts.

Task-level comparisons illustrate possible sources of complementarity. RSRT uniquely solves a system-administration task by resuming after a rejected completion, removing residual processes, and correcting the required log. StateM uniquely solves a multi-requirement software task by enumerating requirements in a contract and catching an exact dependency-version constraint missed by the other harnesses. Terminus 2 uniquely solves a watershed reconstruction task through direct exploration and implementation. These task-level examples illustrate how continued execution, structured checks, and direct exploration can address different failure modes.

Task coverage by domain (Qwen-3.8-27B; at least one passing rollout)

		Terminus 2	RSRT	StateM	Union	
RST	Environment setup (59)	18.6	18.6	15.3	18.6	unique: T0, R0, S0
	System administration (76)	23.7	26.3	21.1	31.6	unique: T2, R5, S1
	Scripting & automation (56)	25.0	35.7	30.4	42.9	unique: T0, R5, S4
	Software dev. & debugging (80)	20.0	25.0	26.2	30.0	unique: T0, R1, S2
	Version control (36)	33.3	30.6	25.0	38.9	unique: T3, R0, S0
	Containers & cloud (37)	18.9	18.9	16.2	27.0	unique: T2, R1, S1
	Security & networking (41)	17.1	19.5	14.6	26.8	unique: T1, R2, S2
	Data & databases (30)	26.7	33.3	33.3	33.3	unique: T0, R0, S0
	Other (5)	20.0	20.0	20.0	20.0	unique: T0, R0, S0
SWR	Media & signal (413)	11.1	13.1	10.4	22.3	unique: T18, R22, S12
	Earth & geospatial (475)	12.2	8.6	7.2	19.8	unique: T28, R18, S15
	Life sciences & chemistry (389)	7.5	2.3	3.1	10.5	unique: T20, R5, S7
	Physical sci. & simulation (384)	24.2	22.1	16.4	30.7	unique: T16, R12, S5
	CAD, 3D & EDA (198)	41.9	38.9	37.9	42.9	unique: T2, R1, S0
	Systems & infrastructure (298)	15.1	9.7	10.1	25.8	unique: T27, R16, S16
	Data, query & formal (251)	25.9	22.7	22.7	28.3	unique: T10, R3, S3
	Documents & web (101)	51.5	51.5	51.5	51.5	unique: T0, R0, S0

● best single harness in the row
 Exclusive solves: T = Terminus 2; R = RSRT; S = StateM

Figure 4 Different harness covers differnt domains of expertise. Cell values report the percentage of tasks with at least one passing rollout, using all recorded rollouts on tasks evaluated by all three harnesses. A red marker identifies the strongest individual harness in each row. The final column reports the union, and the right-hand annotations report tasks solved exclusively by Terminus 2 (T), RSRT (R), or StateM (S).

3.4 Recursive Self-Rewrite and Case Study

Using diverse harnesses broadens the successful experiences available to a single model. We then rewrite these experiences for learning under the general Terminus 2 harness using Qwen-3.8-27B in the planner, critic, and executor roles. Specifically, we reconstruct successful rollouts from StateM and Recursive Self-Reflect Terminus so that their useful procedures are expressed through the general Terminus 2 interface. We sample $K = 4$ candidate runbooks, screen them with the critic, and use each retained runbook to guide $M = 4$ fresh executions under the general harness at temperature 0.7 for each successful source rollout,. We keep only passing rewrites that satisfy the filtering requirements in Section 2, remove private runbooks and critic conversations, and use the remaining public execution histories as training trajectories. We collect a total of 12,893 rollouts with RECURSIVE SELF-REWRITE covering 975 unique tasks, with passing rewrites of a total 86%, a total of available passing 11,094 rewrites.

We examine two rewritten tasks to show how a source experience can be rewritten into new trajectories under the general Terminus 2 harness: Markdown inline parsing from Recursive Self-Reflect Terminus and OpenFOAM PitzDaily reconstruction from StateM. Figure 5 shows a passing and a failing execution guided by the same runbook in each case, with representative command excerpts from the source and both rewrites. These are independent sampled executions. We count parsed assistant turns, including command-free turns,

	Terminus 2	RSRT	StateM
Passing trajectories	766	599	636
Turns (mean)	19.4	23.2	24.0
Turns (median)	17.0	18.0	24.0
Commands per turn	2.10	1.96	2.37
Completion claims per trajectory	1.72	2.20	1.11
Passes after rejection (%)	0.0	4.5	0.0
Exploration (%)	55.5	50.6	35.3
Python execution (%)	21.8	22.1	12.6
State management (%)	0.0	0.0	29.6
File writing (%)	9.0	8.6	5.7
Testing and verification (%)	5.0	5.1	4.0

Table 3 Execution styles on passing trajectories. All passing rollouts on the common task set are included. Command categories are assigned by the audit’s rule-based classifier and reported as percentages of issued commands.

consistently across sources and rewrites.³

Case 1: Markdown inline parsing

RSRT source; fresh rewrites under general Terminus 2

7/12 pass (58.3%)

64 -> 32 median turns

RSR Find a compatible parser > Match the output summary > Probe the table parameter

Sample trajectory excerpts

Source: 64 turns

Turn 26: try generation
tree-sitter generate

Turn 51: use prebuilt parser
tree-sitter parse -l ...node
--lang-name markdown /tmp/test.md

Pass: 32 turns

Turn 9: use prebuilt parser
tree-sitter parse -l ...node
--lang-name markdown /tmp/test.md
Turn 21: check reconstruction
python3 /app/tools/feedback

Fail: 38 turns

Turn 26: inspect reference bytes
xxd .../parse.txt | head -20
bash: xxd: command not found
Turn 36: check after revision
python3 /app/tools/feedback

Case 2: Reconstructing OpenFOAM demo outputs

StateM source; fresh rewrites under general Terminus 2

7/8 pass (87.5%)

38 -> 29 median turns

RSR Identify varying fields > Build a summary generator > Check public scenarios

Sample trajectory excerpts

Source: 38 turns

Turn 25: validate contract
python3 .../tb_receipt.py
validate-contract
Turn 26: advance workflow
statem goto implement

Pass: 28 turns

Turn 26: inspect implementation
head -20 .../reconstruct.py
Turns 27-28: rebuild and check
python3 /tmp/build.py
python3 /app/tools/feedback

Fail: 36 turns

Turn 11: inspect summary fields
python3 -c "..."
NameError: name 'kind' is not defined
Turn 12: inspect summary schema
python3 << 'PYEOF' ... PYEOF
KeyError: 'line_count'

Commands shortened for display; ellipses omit paths or bodies. Shown errors do not establish the final failure cause.

Figure 5 From source experience to successful and unsuccessful rewrites. Each case compares a source trajectory with a passing and a failing rewrite guided by the same runbook. Headers report success rates and median passing lengths across all recorded rewrites (12 Markdown; eight OpenFOAM). Commands are abbreviated; displayed errors do not establish the cause of final verifier failure. Verifier success does not imply quality-audit acceptance.

Markdown task: earlier setup does not guarantee successful reconstruction. For the Markdown inline-parsing task, the source trajectory spends many turns on setup, package compatibility, repeatedly calling the same tool

³The parsed sources contain 64 and 38 turns, respectively; their registry entries report 65 and 39. We use the parsed histories for both the traces and length comparisons.

across multiple turns, and parser selection before reaching a workable solution. Rewritten executions avoid much of this early exploration by directly reusing guidance about the compatible parser and key reconstruction steps. Even so, success is not guaranteed: some rewrites pass while others fail after similarly short executions. Across 12 rewrites, 7 pass (58.3%). The median passing rewrite is 32 turns, a 50.0% reduction from the 64-turn source. However, this case shows that shortening the setup phase does not remove the core reconstruction difficulty.

OpenFOAM: fresh execution retains analysis and repair. For the OpenFOAM PitzDaily task, the source trajectory interleaves analysis, implementation, and structured checking. Rewritten executions under the general harness preserve this pattern, which they continue to analyze outputs, revise implementations, and repair errors during execution, but they do so without issuing source-harness-specific commands. Across 8 rewrites, 7 pass (87.5%). The median passing rewrite is 29 turns, although some successful rewrites are still longer than the source. This case shows that rewriting can preserve useful analysis and repair behavior while shifting execution to the general interface.

These cases show that runbooks can transfer useful procedures into fresh executions under a general harness. They also show that the same guidance can still lead to different outcomes, so shorter or cleaner executions do not by themselves guarantee success.

3.4.1 Command Similarity Across Rewrites

We compare pairs of rewrites guided by the *same runbook* with pairs guided by *different runbooks* for each task in Figure 5, including both passes and failures. Markdown has 12 rewrites from three runbooks, yielding 18 same-runbook and 48 different-runbook pairs. OpenFOAM has eight rewrites from two runbooks, yielding 12 and 16 pairs. We average each metric separately within these groups.

Metrics. We use several trajectory-structure metrics to compare executions. Exact command overlap is the Jaccard similarity between sets of recorded command fingerprints, where shell whitespace is normalized but script bodies remain distinct. Command-order similarity is defined as $2L/(n_i + n_j)$, where L is the longest common subsequence of command fingerprints and n_i, n_j are the command counts. Tool-set overlap compares the sets of invoked program names. Tool-transition overlap applies Jaccard similarity to distinct ordered pairs of adjacent commands’ tool sets. Action-sequence similarity is defined as $1 - d/\max(n_i, n_j)$, where d is the unit-cost Levenshtein distance between sequences of heuristic action labels such as exploration, Python execution, and verification. Commands are ordered across turns. All scores are scaled to 0–100 and measure execution structure rather than semantic equivalence.

Similarity metric	Markdown (12 rewrites)		OpenFOAM (8 rewrites)	
	Same Runbook	Different Runbook	Same Runbook	Different Runbook
Pairs compared	18	48	12	16
Exact command overlap (Jaccard)	19.1	12.1	12.6	10.4
Command order (normalized LCS)	24.6	15.7	18.2	14.7
Tool-set overlap (Jaccard)	66.9	52.9	83.3	84.4
Tool-transition overlap (Jaccard)	34.0	18.9	28.4	28.5
Action-sequence similarity (edit)	53.6	42.0	61.0	63.1
Exact overlap, omit first two turns	12.5	6.2	7.2	4.7

Table 4 Command similarity within and across runbooks. Same pairs share a runbook; different pairs use distinct runbooks for the same task. Scores are mean pairwise similarities on a 0–100 scale, including both passing and failing rewrites. Higher means more similar. The final row excludes each rewrite’s first two assistant turns.

Shared runbooks increase command overlap. For Markdown, trajectories from the same runbook show higher exact command overlap (19.1% vs. 12.1%) and command-order similarity (24.6% vs. 15.7%) than trajectories from different runbooks (Table 4). OpenFOAM shows the same pattern, though more weakly: 12.6% vs. 10.4% for exact overlap and 18.2% vs. 14.7% for command order. After excluding the first two turns, within-runbook exact overlap remains higher in both cases.

Workflow similarity is more task-dependent. Markdown also shows higher within-runbook tool-transition overlap (34.0% vs. 18.9%) and action-sequence similarity (53.6% vs. 42.0%). OpenFOAM shows little difference in tool transitions (28.4% vs. 28.5%), and action-sequence similarity is slightly lower within runbooks (61.0% vs. 63.1%). Thus, shared runbooks are associated with more similar command-level behavior, while broader workflow similarity depends more on the task.

3.5 Model Training and Evaluation

We now test the usefulness of the reconstructed experience trajectories on training it into the model. We compare three groups based on the same Qwen-3.8-27B model. **Base** is the original model. **Direct SFT** fine-tunes the model directly on the passing source rollouts collected with the discovery harnesses after removing harness-specific insertion phrases and parts, without runbook reconstruction or fresh re-execution. **Recursive Self-Rewrite** fine-tunes the model on verified rewrite trajectories, in addition to the 766 direct passing trajectories from Terminus 2 without rewriting.

Evaluation benchmarks. We evaluate on Terminal-Bench 2 (TB2) [19], Terminal-Bench 3 (TB3), Terminal-Bench 4 (TB4) [18], Terminal-Bench Hard (TBH) [11], Long-Horizon Terminal Bench (LHTB) [13], and Software Terminal 100 (SWR 100).⁴

3.6 Results and Discussion

Table 5 reports both pass@3 and the mean per-run pass rate over three runs for TB2, TB3, TB4, TBH, and SWR100. Pass@3 measures success within three attempts, while the mean averages the three individual-run pass rates. LHTB is evaluated using process reward. Parenthesized pass@3 counts are estimated from the reported percentages and benchmark sizes by rounding to the nearest integer; LHTB counts are reported task completions. RSR achieves higher pass@3 and mean per-run pass rate on all five benchmarks, as well as higher LHTB process reward, than Direct SFT.

Table 5 RSR achieves the best results across benchmarks. We report pass@3 and the mean per-run pass rate over three runs for TB2, TB3, TB4, TBH, and SWR100, and process reward for LHTB. In addition, Direct SFT performs worse on TB 2 after training due to circular and deadend behaviors in raw trajectories from RSR.

Benchmark	Metric	Base	Direct SFT	RSR
TB 2	Pass@3	57.0% (51/89)	53.4% (48/89)	74.2% (66/89)
TB 3	Pass@3	0.0% (0/74)	5.4% (4/74)	9.5% (7/74)
TB 4	Pass@3	1.5% (1/66)	4.5% (3/66)	9.1% (6/66)
TBH (ours)	Pass@3	39.0% (39/100)	56.0% (56/100)	63% (63/100)
SWR100 (ours)	Pass@3	3.0% (3/100)	3.0% (3/100)	6.0% (6/100)
TB 2	Mean (3 runs)	51.7%	43.8%	70.1%
TB 3	Mean (3 runs)	0.0%	1.4%	6.8%
TB 4	Mean (3 runs)	0.0%	3.2%	6.1%
TBH (ours)	Mean (3 runs)	31.0%	42.5%	54.0%
SWR100 (ours)	Mean (3 runs)	3.0%	3.0%	5.0%
LHTB (ours)	Process reward	0.21 (0/46)	0.25 (0/46)	0.29 (0/46)

⁴Software Terminal is a self-curated benchmark.

Diverse harness experiences still support model improvement. In pass@3, Direct SFT improves over Base by 17.0 percentage points on TBH, 5.4 points on TB 3, and 3.0 points on TB 4. It matches Base on SWR100, but falls by 3.6 points on TB 2. Its mean per-run pass rate on TB 2 also falls, from 51.7% to 43.8%, a decrease of 7.9 percentage points. These mixed results suggest that diverse source experiences can improve performance even without rewriting. However, direct SFT can also introduce undesirable behavior shifts. In particular, Recursive Self-Reflect Terminus produces many trajectories longer than 100 steps, which may be difficult for a 27B model to absorb without the original sustained-execution support. In our inspection of three trajectories from the direct-SFT checkpoint, we observed a recurring **dead end** behavior: the model loops around the same action pattern without making real progress, which ultimately causes task failure.

Rewriting improves experience scaling. By scaling successful experiences and trajectories, and rewriting them under a standardized harness, RSR further improves over Direct SFT by 20.8, 4.1, 4.6, 7.0, and 3.0 percentage points in pass@3 on TB 2, TB 3, TB 4, TBH, and SWR100, respectively. The mean per-run results also favor RSR: gains over Direct SFT are 26.3, 5.4, 2.9, 11.5, and 2.0 percentage points on the same five benchmarks, respectively. On TB 2, RSR reaches 74.2% pass@3 and 70.1% mean per-run success; on TBH, it reaches 63.0% and 54.0%, respectively. These results suggest that expanding experience and standardizing it under a general harness can produce better training samples for generalization. The benefit also extends to process reward. On LHTB, process reward increases from 0.21 for Base and 0.25 for Direct SFT to 0.29 for RSR. All three groups still complete zero of the 46 tasks, so this gain reflects greater partial progress rather than more fully solved tasks.

3.7 Ablation Studies of Harnesses on Sample Benchmarks

To further examine whether different harnesses expose different problem-solving strengths on benchmarks, we evaluate Qwen-3.8-27B base model under Terminus 2, StateM, and Recursive Self-Reflect Terminus on TB2, TBH, and TB3. This comparison examines the effect of changing the execution harness, complementing the training comparison in Section 3.6 on benchmarks without training. Here, RSRT denotes the discovery harness from Section 3.2, while RSR denotes the rewriting method. We run once on each benchmark using each harness and show results in Table 6.

Table 6 Different harnesses also affect Qwen-3.8-27B Base Model performance on different benchmarks. All rows use the same base model without additional training. Entries show reported success rates (%) with the supplied task counts in parentheses.

Harness	TB2 (89)	TBH (100)	TB3 (74)
Terminus 2	51.7 (46)	33.0 (33)	0.0 (0)
Recursive Self-Reflect Terminus	55.1 (49)	66.0 (66)	4.1 (3)
StateM	58.4 (52)	46.0 (46)	1.3 (1)
Union of all three	68.5 (61)	70.0 (70)	4.3 (3)

Smaller models can solve challenging tasks under stronger harnesses. For the Qwen-3.8-27B base model, TBH and TB3 remain challenging benchmarks, and the set of passing tasks varies substantially across harnesses. This further suggests that different harnesses can unlock different problem-solving behaviors even for the same model. StateM, by contrast, is better aligned with TB2 and achieves the highest pass@1 on that benchmark. This shows that the same base model can solve more challenging tasks when paired with a more suitable harness, even without additional training. RECURSIVE SELF-REWRITE builds on this observation by collecting more such successful trajectories and using them to further improve the model through training.

4 Related Work

Harness engineering and optimization. A growing line of work improves agent performance by changing the execution harness while keeping model weights fixed. Harnesses structure planning, tool use, progress tracking, verification, and recovery, helping models complete goals and sustain long-horizon tasks that they struggle to solve under less structured execution [19, 26, 33]. Terminal-Bench highlights this dependence by evaluating executable command-line tasks, where success reflects both the model and its interaction system [19]. Terminus-KIRA, StateM, Deep Agents, Ledger, Meta-Harness, Agentic Harness Engineering, Self-Harness, and AutoSaddler improve this system through stronger verification, continuation, workflow control, or automated harness search [8, 10, 14, 19, 24, 26, 31, 39]. Harness changes avoid model retraining, they offer a relatively inexpensive route to higher solve rates and can make weaker models more competitive. Better harnesses can even reverse model rankings on particular benchmarks, enabling a weaker or lower-cost model to outperform a stronger model operating without the same support [16, 26]. Self-Harness improves fixed models on Terminal-Bench by changing only the harness, raising MiniMax M2.5 from 40.5% to 61.9% and Qwen3.5-35B-A3B from 23.8% to 38.1% on its held-out split [39]. StateM similarly reports large gains on Terminal-Bench 2.1, including improvements for GPT-5.6 Luna and DeepSeek-V4 Flash [26]. Beyond terminal tasks, AutoHarness and AI4AI at Test-Time show that stronger external control can substantially improve weaker models without parameter updates [16, 25]. Task-CoEvolve further reduces the cost of harness optimization through adaptive validation-task selection [20]. Controlled studies such as the Scaffold Effect and Harness-Bench show that harness choice affects both solve rate and efficiency, motivating its inclusion in model-performance reporting [32, 36]. Our complementary objective is not to optimize a single harness, but to use those diverse harnesses as discovery tools and maximize a weak models’ ability to solve more tasks.

Recursive self-improvement and Self-Evolving of agent systems. Recursive self-improvement work uses past execution results to iteratively improve an agent’s model, harness, or both [6, 7, 12, 41]. ModularRSI [34] and SoL-Pi [15] analyze trajectories to revise harness modules or improve execution efficiency, while Dream-RSI optimizes exploration-policy code through replay of recorded discovery trees [41]. ScienceBuddy couples these ideas more tightly by adapting the harness in an inner loop and training the model in an outer loop [35]. These methods improve the execution system itself, or co-evolve the model and harness together. In contrast, RECURSIVE SELF-REWRITE does not optimize a single harness or assume an autonomous recursive cycle. Instead, it collects successful solutions from multiple harnesses and reconstructs them as verified demonstrations under a shared general interface, so that the model can improve through supervised learning from its own harness-assisted experience.

Learning from trajectories and harness-supported behaviors. Another related line of work studies how execution experience can be reused as contextual guidance or as direct training signal. Reflexion and ExpeL retain reflective lessons from earlier attempts and retrieve them during later problem solving without updating model weights [29, 40]. SWE-Gym provides executable software-engineering tasks and verified successful trajectories for policy learning [23]. Other work studies how behaviors initially supported by external guidance can become part of the model itself, including SKILL0, PATS, and EvoHarness-RL [17, 21, 28]. Cross-harness training work further shows that trajectory compatibility matters: DCAS studies transfer across scaffolds, while Yu et al. [38] show that directly imitating stronger experts can misfit a weaker model’s execution setting [30]. Harness-Zero most directly studies transferring harness-supported behaviors into model parameters for deployment under a fixed target harness [37]. RECURSIVE SELF-REWRITE aims to preserve useful behaviors after specialized support is removed. Our focus is on reconstructing completed successes from diverse harnesses into verified demonstrations under a shared general harness with model’s own ability.

5 Conclusion

RECURSIVE SELF-REWRITE improves a model through its own experiences collected under diverse problem-solving harnesses while generating more valuable problem-solving trajectories. Different harnesses help the

same model discover more complementary successful solutions, but their trajectories often contain harness-specific execution conventions that do not transfer directly to a general harness. RSR addresses this gap by reconstructing those successful experiences into new verified trajectories under a general harness. Our analysis and experiments show that diverse harnesses expand the set of tasks a model can solve, and that rewriting these experiences helps the model relearn useful planning, checking, recovery, and sustained-execution behaviors under a general harness. These results suggest that diverse harnesses are not only effective tools for discovery, but also valuable sources of experience through which models can improve themselves on complex long-horizon tasks that are difficult to solve or annotate at scale.

References

- [1] Anthropic. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>, 2025. Accessed October 1, 2026.
- [2] Anthropic. Effective harnesses for long-running agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, 2025. Accessed October 1, 2026.
- [3] Boyuan Deng, Shuyi Fan, Hongyang Zhang, and Xinhong Xie. Jev for scientific decisions: Evaluating semantic choices and their consequences. *arXiv preprint arXiv:2609.24965*, 2026.
- [4] EverMind AI. Raven: The harness of harnesses for composable agentic intelligence. *arXiv preprint arXiv:2609.33439*, 2026. URL <https://arxiv.org/abs/2609.33439>.
- [5] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, 2026. URL <https://doi.org/10.5281/zenodo.20953922>.
- [6] Yicheng He, Chengsong Huang, Zongxia Li, Jiaxin Huang, and Yonghui Yang. Visplay: Self-evolving vision-language models from images, 2025. URL <https://arxiv.org/abs/2511.15661>.
- [7] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-zero: Self-evolving reasoning llm from zero data. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, editors, *International Conference on Learning Representations*, volume 2026, pages 130770–130790, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/d49b9aacebda61051166335af6fd3061-Paper-Conference.pdf.
- [8] KRAFTON AI and Ludo Robotics. Terminus-kira: Boosting frontier model performance on terminal-bench with minimal harness, 2026. URL <https://github.com/krafton-ai/kira>.
- [9] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [10] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. <https://arxiv.org/abs/2603.28052>, 2026.
- [11] Zhongzhi Li, Yucheng Shi, Zongxia Li, Ruhan Wang, Anhao Li, Zixun Huang, Junyao Yang, Lei Ke, Ninghao Liu, Haitao Mi, et al. Recursive synthesis for long-horizon terminal tasks. *arXiv preprint arXiv:2608.05466*, 2026.
- [12] Zongxia Li, Hongyang Du, Chengsong Huang, Xiyang Wu, Lantao Yu, Yicheng He, Jing Xie, Xiaomin Wu, Zhichao Liu, Jiarui Zhang, and Fuxiao Liu. Mm-zero: Self-evolving multi-model vision language models from zero data, 2026. URL <https://arxiv.org/abs/2603.09206>.
- [13] Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, et al. Long-horizon-terminal-bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading. *arXiv preprint arXiv:2607.08964*, 2026.
- [14] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- [15] Haozhe Liu, Tian Ye, Sensen Gao, Qihang Cao, Yitong Li, Mingchen Zhuge, Duomin Wang, Ruihua Zhang, Ping Luo, Jiawang Bian, et al. Sol-pi: Recursively scaling auto-research loops for efficient agent harness. *arXiv preprint arXiv:2609.20519*, 2026.

- [16] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*, 2026.
- [17] Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.
- [18] Ryan Marten. Terminal-Bench 4.0. Terminal-Bench release announcement, 2026. URL <https://www.tbench.ai/news/terminal-bench-4-0>. Accessed October 1, 2026.
- [19] Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Jenia Jitsev, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Kwesi Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- [20] Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. Task-coevolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*, 2026.
- [21] Xuying Ning, Dongqi Fu, Tianxin Wei, Hanqing Zeng, Yuanchen Bei, Bingxuan Li, Zihao Li, Qifan Wang, Xiang Shen, Yifan Wu, et al. Evoharness-rl: Learning self-evolving runtime harness for long-horizon llm agents. *arXiv preprint arXiv:2608.05446*, 2026.
- [22] OpenAI. Codex. <https://github.com/openai/codex>, 2025. OpenAI coding agent / CLI.
- [23] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.
- [24] Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, et al. Autosaddler: Automatic harness optimization with durable updates from agent execution traces. *arXiv preprint arXiv:2608.23041*, 2026.
- [25] Cheng Qian, Wenting Zhao, Liangwei Yang, Heng Wang, Jieli Qiu, Heng Ji, Silvio Savarese, Huan Wang, and Shelby Heinecke. Ai4ai at test-time: Strong-to-weak capability transfer via harnesses. *arXiv preprint arXiv:2608.12307*, 2026.
- [26] Ziheng Qin, Yaxin Lu, Zhangyang Atlas Wang, and Kai Wang. StateM: Reaching 95.3% raw accuracy, or a \$15 frontier run, on Terminal-Bench 2.1 via harness scaling, 2026. URL <https://arxiv.org/abs/2608.15089>.
- [27] Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026. URL <https://qwen.ai/blog?id=qwen3.8>.
- [28] Yipeng Shi, Zhipeng Ma, Yue Wang, Qitai Tan, Yang Li, Peng Chen, and Zhengzhou Zhu. Pats: Policy-aware training scaffolding for agentic reinforcement learning. *arXiv preprint arXiv:2607.21419*, 2026.
- [29] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [30] Kishanthan Thangarajah, Boyuan Chen, and Ahmed E Hassan. Dcas: Decoupling cli agent scaffolding to internalize planning across scaffolds. *arXiv preprint arXiv:2608.06113*, 2026.
- [31] Vivek Trivedy. Improving deep agents with harness engineering. LangChain engineering blog. <https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering>, February 2026. Published February 17, 2026; accessed September 23, 2026.

- [32] Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation. *arXiv preprint arXiv:2607.22585*, 2026.
- [33] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. In *COLM 2026 The 2nd Workshop on Lifelong Agents: Learning, Aligning, and Evolving*, 2026.
- [34] Siwei Wu, Jincheng Ren, Yizhi Li, Haau-Sing Li, Chengran Yang, Yuxuan Zhang, Weicheng Gu, Jian Yang, Riza Batista-Navarro, Chuanyi Zhang, et al. Modularrsi: Modular and generalizable recursive harness self-improvement. *arXiv preprint arXiv:2609.14857*, 2026.
- [35] Shuhan Xue, Jianyuan Zhong, Ziyuan Nan, Wenbin Li, Zhaochen Yu, Jinchao Ding, Qiang Gao, Pengyu Zhan, Yuntong Zhang, Tian Cheng, et al. Sciencebuddy: Recursive-in-recursive self-improvement for interactive scientific agents. *arXiv preprint arXiv:2609.17523*, 2026.
- [36] Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, et al. Harness-bench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922*, 2026.
- [37] Haoran Ye, Yuxing Lu, Haonan Dong, Zhaochen Su, and Guojie Song. Harness-zero: Harness distillation via agent-as-harness, 2026. URL <https://arxiv.org/abs/2609.24974>.
- [38] Zhou Yu, Bin Bi, Shiva Kumar Pentiyala, Shubham Mehrotra, Sougata Chaudhuri, Shilpa Bhagavath, Zeyuan Chen, Ran Xu, Phil Mui, James Zhu, et al. Co-evolving harnesses and models: On-policy correction helps weaker models catch up where imitation fails. *arXiv preprint arXiv:2609.09134*, 2026.
- [39] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.
- [40] A Zhao, D Huang, Q Xu, M Lin, YJ Liu, and G Huang. Expel: Llm agents are experiential learners. arxiv, 2023.
- [41] Tong Zheng, Xidong Wu, Zheng Zhang, Zhankui He, Chaoyi Zhang, Benjamin Coleman, Ruoqiao Wei, Di Bai, Haolin Liu, Rui Liu, et al. Dream-rsi: Recursive self-improvement through evolving worlds. *arXiv preprint arXiv:2609.14858*, 2026.